

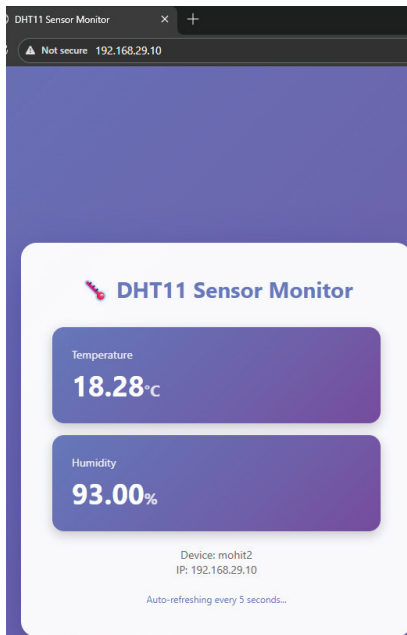


Figure 1: Local server temperature and humidity sensor

identification and starts a local web server on port 80.

Once connected, the device prints its local IP address, which can be accessed from any browser on the same network.

To send sensor data to the remote server (core logic), use the following code:

```
void sendDataToServer(float temp, float humidity)
```

Key operations inside this function are:

- *URL construction*

```
String url = "http://" + serverHost + ":" + serverPort + serverPath;
```

Creates a complete endpoint dynamically, allowing easy server changes.

- *JSON payload creation*

```
{
  "device": "mohit2",
  "temp": 27.45,
  "humidity": 61.20
}
```

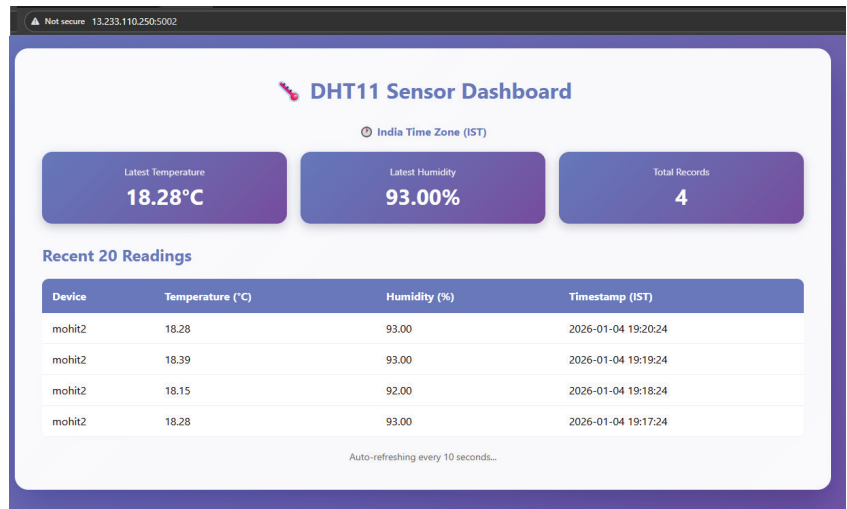


Figure 2: Public server, which can be accessed from any location

JSON is lightweight, human-readable, and widely supported, perfect for REST APIs and cloud ingestion pipelines.

- *HTTP POST transmission*

```
http.POST(jsonPayload);
```

Sends sensor data securely over HTTP. The server can store data in a database, trigger alerts, or feed dashboards.

This pattern is widely used in IoT cloud platforms and private telemetry systems.

- Periodic sensor reading loop is:

```
if (currentTime - lastUploadTime >= uploadInterval)
```

Inside loop(): reads temperature and humidity from DHT11. It validates sensor data (!isnan) and sends data to the remote server at fixed intervals.

This ensures controlled network usage, predictable data sampling, and reliable long-term operation.

- Local web server for real-time monitoring is:

```
WiFiServer server(80);
```

The ESP8266 also acts as a mini HTTP server.

When a client connects:

- Sensor data is read instantly.
- Response is returned in JSON format:

```
{
  "temp": 28.10,
  "humidity": 60.50
}
```

This matters because it:

- Enables offline or LAN-only monitoring.
- Is useful for diagnostics, maintenance, or edge applications.
- Reduces dependency on cloud connectivity.

This IoT system can work in both online and offline conditions, making it reliable and flexible. The use of fixed time intervals and simple JSON data makes it easy to manage and understand. This approach is useful for smart homes, small industries, and basic monitoring systems. **END** 🐧

By: Dr Mohit

The author is a CEH and ECSA, and has worked at IBM, Teramatrix and Sapient. He has written the books titled 'Python Penetration Testing Essential', 'Python: Penetration Testing For Developers', and 'Learn Python in 7 Days'.